

Universidad de San Carlos de Guatemala

Centro Universitario de Occidente

División Ciencias de la Ingeniería

Software Avanzado

Eiler Rigoberto Gómez Figueroa

202031693

Ensayo Saga vs Circuit Breaker



Ing. Moises Granados

Quetzaltenango, Octubre del 2026

Saga vs Circuit Breaker

Cuando una aplicación se construye como un monolito con una sola base de datos, garantizar que una operación se complete por entero o no se realice en absoluto es relativamente sencillo: basta con envolver todo en una transacción de base de datos. Si algo falla a mitad de camino, el motor revierte los cambios y el sistema queda como estaba.

En una arquitectura de microservicios esa garantía desaparece. Cada servicio tiene su propia base de datos, y una operación de negocio como completar una compra puede tocar cuatro servicios distintos. No existe un motor central que pueda deshacerlo todo. Y además, los servicios se comunican por red, lo que significa que cualquier llamada puede fallar, demorarse o quedarse colgada.

Estos dos problemas, la pérdida de atomicidad y la fragilidad de la comunicación remota, dieron origen a dos patrones distintos: el Saga Pattern y el Circuit Breaker Pattern. Estos se confunden con frecuencia porque ambos aparecen en el mismo contexto, pero resuelven cosas diferentes. Este ensayo analiza qué hace cada uno, cómo funcionan, y por qué a veces en un sistemas reales se usan juntos en lugar de elegir entre ellos.

1. El problema de las transacciones distribuidas

El enfoque tradicional para coordinar cambios en varias bases de datos se llama Two-Phase Commit, o 2PC. Consiste en un coordinador que pregunta a todos los participantes si pueden confirmar, y solo entonces les ordena hacerlo. Funciona, pero tiene un costo alto: mientras dura el proceso, todos los recursos quedan bloqueados.

En microservicios ese costo es inaceptable. Los recursos bloqueados durante llamadas de red degradan el rendimiento, y si el coordinador falla, todos los participantes quedan esperando indefinidamente. Además, 2PC obliga a que todos los servicios usen tecnologías compatibles, lo cual contradice la autonomía tecnológica que motivó adoptar microservicios en primer lugar.

La alternativa es renunciar a la atomicidad inmediata y aceptar la consistencia eventual: el sistema puede estar temporalmente inconsistente, pero converge a un estado válido. Esa es la premisa del Saga Pattern.

2. Saga Pattern: consistencia sin transacciones globales

Chris Richardson, referente principal en patrones de microservicios, define la saga de forma directa:

"Una saga es una secuencia de transacciones locales. Si una transacción local falla porque viola una regla de negocio, la saga ejecuta una serie de transacciones compensatorias que deshacen los cambios realizados por las transacciones anteriores." (Richardson, microservices.io)

La idea central es que cada servicio ejecute su propia transacción local, que sí es atómica dentro de su base de datos, y publica el resultado. Si algún paso falla, en lugar de un rollback automático se ejecutan operaciones explícitas que revierten lo hecho.

Aquí hay un matiz importante que suele pasarse por alto: una compensación no es un rollback. Un rollback borra la historia; una compensación añade un evento nuevo que anula el efecto del anterior. Si se cobró al cliente, la compensación no es borrar el cobro sino emitir

un reembolso. El cobro ocurrió y quedó registrado. La saga ofrece una garantía de negocio, no una garantía técnica.

Existen dos formas de coordinar una saga. En la coreografía, cada servicio escucha eventos y reacciona publicando los suyos, sin coordinador central. En la orquestación, un componente dedicado conoce la secuencia completa y va indicando a cada participante qué hacer.

Richardson señala que la coreografía se vuelve difícil de seguir cuando hay muchos participantes, porque los eventos rebotan entre servicios sin que exista un lugar donde leer el flujo completo. La orquestación concentra esa lógica en un solo punto, lo que la hace más observable y testeable. La recomendación práctica habitual es usar coreografía para flujos de dos o tres servicios, y orquestación a partir de ahí.

El Saga Pattern tiene costos. La consistencia eventual significa que existen ventanas de tiempo donde los datos son inconsistentes, y el código de compensación debe escribirse y probarse igual que el camino feliz. Además hay que resolver qué ocurre si una compensación falla, escenario que ningún patrón resuelve automáticamente y que normalmente requiere intervención humana.

3. Circuit Breaker Pattern: evitar el fallo en cascada

El segundo problema es distinto. No se trata de consistencia de datos sino de disponibilidad. Martin Fowler lo plantea así:

"Una de las grandes diferencias entre las llamadas en memoria y las llamadas remotas es que las llamadas remotas pueden fallar, o quedarse colgadas sin respuesta hasta que se alcanza algún límite de tiempo." (Fowler, martinfowler.com)

El escenario peligroso es el siguiente: un servicio lento acumula llamadas pendientes; cada llamada consume un hilo del servicio que la invoca; esos hilos se agotan; el servicio invocador deja de responder; y quien depende de él empieza a fallar también. Un solo componente degradado puede derribar el sistema completo. Eso se llama fallo en cascada.

El patrón fue popularizado por Michael Nygard en su libro *Release It!* (2007), y posteriormente difundido por Fowler y por Netflix con su librería Hystrix. La analogía viene de la electricidad: un interruptor que corta el paso de corriente antes de que el exceso dañe la instalación.

El Circuit Breaker es una máquina de estados con tres posiciones. En estado Closed, todas las llamadas pasan normalmente mientras el componente lleva registro de los fallos. Si la tasa de fallos supera un umbral configurado, transita a Open. En estado Open, las llamadas se rechazan de inmediato sin siquiera intentar contactar al servicio: esto se conoce como fallar rápido, y es lo que protege al sistema. Tras un tiempo de espera configurable, pasa a Half-Open, donde permite un número limitado de llamadas de prueba. Si esas llamadas tienen éxito, vuelve a Closed; si fallan, regresa a Open y reinicia el temporizador.

Un detalle que conviene precisar: el estado Half-Open solo se alcanza desde Open. No existe transición directa de Closed a Half-Open.

La utilidad de fallar rápido es doble. Por un lado, libera recursos del servicio invocador, que ya no espera respuestas que no llegarán. Por otro lado, le da al servicio caído margen para recuperarse sin seguir recibiendo tráfico que no puede atender.

El Circuit Breaker rara vez se usa solo. Suele combinarse con Timeout, que limita cuánto se espera una respuesta; Retry, que reintenta operaciones fallidas antes de contarlas como definitivas; Bulkhead, que aísla los recursos asignados a cada dependencia para que la saturación de una no afecte a las demás; y Fallback, que define una respuesta alternativa cuando el circuito está abierto.

4. Comparación: patrones complementarios, no alternativos

Presentar estos dos patrones como un versus puede inducir a error, porque operan en planos distintos. El Saga Pattern responde a la pregunta de qué hacer cuando una operación de negocio no puede completarse; el Circuit Breaker responde a qué hacer cuando un servicio del que dependemos no está respondiendo.

El primero trabaja sobre el estado de los datos y garantiza consistencia eventual. El segundo trabaja sobre el tráfico de red y garantiza disponibilidad. El primero actúa después del fallo, revirtiendo; el segundo actúa antes, impidiendo que las llamadas se hagan.

En un sistema real, consideremos el flujo de compra del presente proyecto: Order, Payment, Inventory y Shipping. La saga define que si Inventory no tiene stock, hay que reembolsar el pago y cancelar la orden. El Circuit Breaker define que si Inventory lleva varias llamadas fallando, no se le sigan enviando peticiones.

La interacción entre ambos es donde está el diseño interesante. Cuando el circuito hacia Inventory se abre, la saga recibe un fallo inmediato en lugar de una espera larga, y puede iniciar la compensación más rápido. El Circuit Breaker convierte un fallo lento en uno rápido, y eso beneficia directamente a la saga: menos tiempo en estado inconsistente, menos recursos bloqueados.

También plantea una decisión de diseño: si el circuito está abierto porque el servicio está temporalmente caído, no porque la operación sea inválida, quizá no convenga compensar de inmediato sino encolar la petición para reintentarla. Distinguir entre un fallo de negocio, “no hay stock” y un fallo de infraestructura “el servicio no responde” es clave, porque el primero exige compensación y el segundo podría resolverse con paciencia.

5. Cuándo aplicar cada uno

El Saga Pattern se justifica cuando existe un proceso de negocio que abarca varios servicios con bases de datos independientes, y cuando una inconsistencia parcial tendría consecuencias reales: cobros sin producto, inventario reservado que nadie va a comprar, envíos programados para órdenes canceladas. No se justifica cuando la operación involucra un solo servicio, donde una transacción local basta.

El Circuit Breaker se justifica en cualquier llamada de red hacia una dependencia que pueda fallar o degradarse, especialmente si esa dependencia es externa o si el servicio invocador atiende tráfico de usuarios. No se justifica en llamadas locales dentro del mismo proceso, ni en operaciones que ya son asíncronas y desacopladas mediante colas, donde la naturaleza del canal ya provee el desacople.

Conclusiones

- El Saga Pattern y el Circuit Breaker Pattern surgen del mismo contexto “la arquitectura distribuida” pero atacan problemas ortogonales. Compararlos como

alternativas sería como comparar un cinturón de seguridad con un sistema de frenos: ambos son mecanismos de seguridad, pero ninguno sustituye al otro.

- El Saga Pattern acepta que en un sistema distribuido no se puede tener atomicidad inmediata, y ofrece a cambio consistencia eventual mediante compensaciones explícitas. Su costo es la complejidad de escribir y probar esas compensaciones, y la necesidad de razonar sobre estados intermedios inconsistentes.
- El Circuit Breaker acepta que las llamadas remotas fallan, y ofrece a cambio contención del daño mediante el fallo rápido. Su costo es menor en implementación (librerías maduras lo resuelven con configuración) pero exige decidir qué hacer cuando el circuito se abre, y esa decisión es de negocio, no técnica.
- Para un ingeniero de software la lección práctica es que ambos deben estar presentes en cualquier arquitectura de microservicios seria. La saga sin circuit breakers pasará largos periodos esperando servicios caídos antes de compensar. Los circuit breakers sin saga protegerán la disponibilidad pero dejarán datos inconsistentes cuando una operación se interrumpa a medias. Juntos cubren las dos dimensiones del problema: que el sistema siga en pie, y que los datos tengan sentido.

Referencias

- Richardson, C. *Pattern: Saga*. Microservices.io.
<https://microservices.io/patterns/data/saga.html>
- Richardson, C. (2018). *Microservices Patterns: With Examples in Java*. Manning Publications.
- Fowler, M. (2014). *CircuitBreaker*. martinowler.com.
<https://martinfowler.com/bliki/CircuitBreaker.html>
- Nygard, M. T. (2018). *Release It! Design and Deploy Production-Ready Software* (2.^a ed.). Pragmatic Bookshelf.
- Garcia-Molina, H., & Salem, K. (1987). *Sagas*. ACM SIGMOD Record, 16(3), 249–259.
<https://www.cs.cornell.edu/andru/cs711/2002fa/reading/sagas.pdf>
- Resilience4j. *CircuitBreaker. Documentación oficial*.
<https://resilience4j.readme.io/docs/circuitbreaker>
- Microsoft. *Circuit Breaker pattern*. Azure Architecture Center.
<https://learn.microsoft.com/en-us/azure/architecture/patterns/circuit-breaker>